

how do i restart the laptop? - G x (5) WhatsApp x Testleaf GenAI API Platform x + Ask Gemini

api.testleaf.com/finetuning

Fine Tuning

MENU

- Usage
- Cost
- API Keys
- Fine-tuning**
- Storage

AGENTS

- Test Case Generator
- Playwright Converter
- Settings
- Profile
- Logout

ftgpt-4o-mini-2024-07-18:testleaf-2::Dj6N1fV5

Status **Succeeded**

Job ID ftjob-Mfd8tSyXuEqZRhjGNFLmCAyg ⓘ

Training Method supervised

Base Model gpt-4o-mini-2024-07-18

Output Model ftgpt-4o-mini-2024-07-18:testleaf-2::Dj6N1fV5

Created At 24 May 2026 22:18:09

Trained Tokens 26,645

Epochs 5

Batch Size 1

LR Multiplier 0.52

Seed 1521445986

Re-run as Job

Close

26,645
TRAINED TOKENS

+ Create

TRAINED TOKENS	CREATED	
26,645	24 May 2026 22:18	ⓘ

1-1 of 1 < 1 > 10 / page

how do i restart the laptop? - G x (5) WhatsApp x Testleaf GenAI API Platform x + Ask Gemini

api.testleaf.com/finetuning

Fine Tuning

MENU

- Usage
- Cost
- API Keys
- Fine-tuning**
- Storage

AGENTS

- Test Case Generator
- Playwright Converter
- Settings
- Profile
- Logout

ftgpt-4o-mini-2024-07-18:testleaf-2::Dj6N1fV5

- 22:19:11 — Files validated, moving job to queued state
- 22:19:12 — Fine-tuning job started
- 22:23:39 — Evaluating model against our usage policies
- 22:23:39 — New fine-tuned model created
- 22:23:39 — Checkpoint created at step 40
- 22:23:39 — Checkpoint created at step 30
- 22:35:44 — Usage policy evaluations completed, model is now enabled for sampling
- 22:35:53 — The job has successfully completed

Re-run as Job

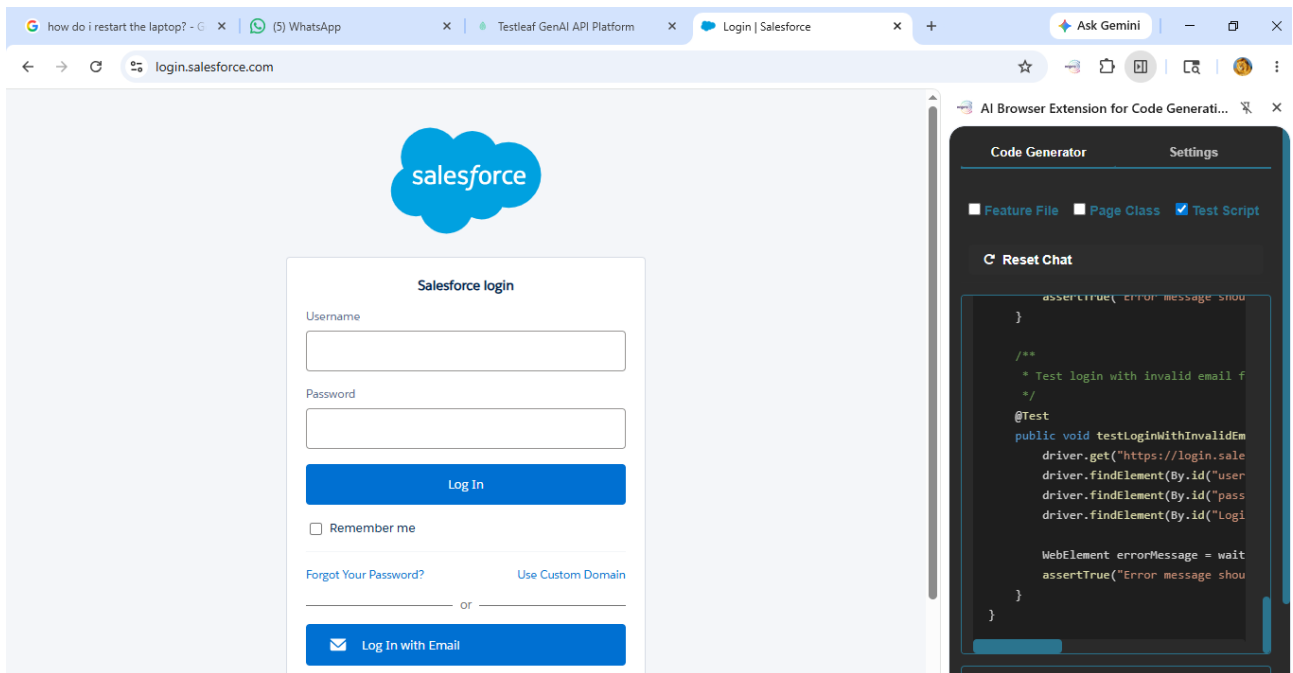
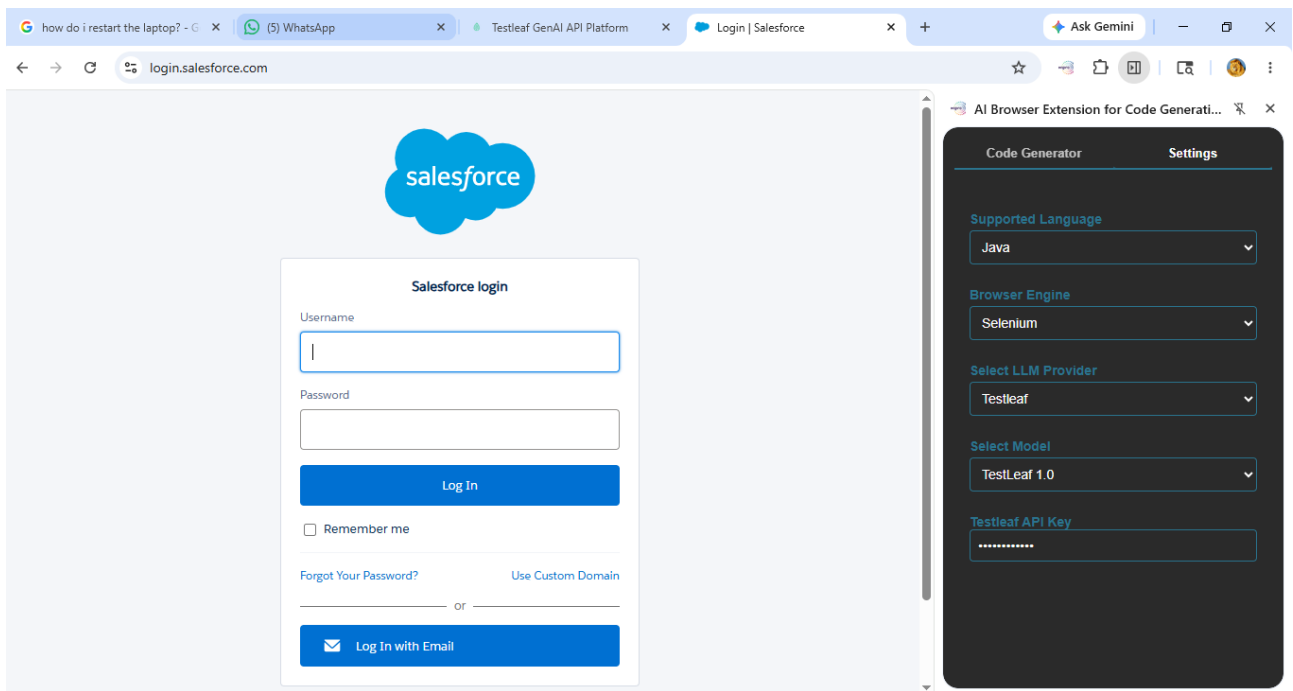
Close

26,645
TRAINED TOKENS

+ Create

TRAINED TOKENS	CREATED	
26,645	24 May 2026 22:18	ⓘ

1-1 of 1 < 1 > 10 / page



Code Generated:

```

package com.testleaf.tests;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

```

```
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import static org.junit.Assert.assertTrue;

/**
 * Selenium Test Class for Salesforce Login Functionality
 */
public class SalesforceLoginTests {
    private WebDriver driver;
    private WebDriverWait wait;

    /**
     * Setup method - Initializes WebDriver and waits
     */
    @Before
    public void setUp() {
        driver = new ChromeDriver();
        wait = new WebDriverWait(driver, 10);
        driver.manage().window().maximize();
    }

    /**
     * Teardown method - Closes the browser
     */
    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }

    /**
     * Test successful login with valid credentials
     */
    @Test
    public void testSuccessfulLogin() {
        driver.get("https://login.salesforce.com/");

wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username"))).sendKeys("admin");

        driver.findElement(By.id("password")).sendKeys("admin123");
    }
}
```

```

        driver.findElement(By.id("Login")).click();

        WebElement successMessage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("success")
));

        assertTrue("Success message should be displayed",
successMessage.isDisplayed());
    }

    /**
     * Test login with invalid credentials
     */
    @Test
    public void testLoginWithInvalidCredentials() {
        driver.get("https://login.salesforce.com/");
        driver.findElement(By.id("username")).sendKeys("invaliduser");
        driver.findElement(By.id("password")).sendKeys("wrongpass");
        driver.findElement(By.id("Login")).click();

        WebElement errorMessage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("error")))
;

        assertTrue("Error message should be displayed",
errorMessage.isDisplayed());
    }

    /**
     * Test login with empty username
     */
    @Test
    public void testLoginWithEmptyUsername() {
        driver.get("https://login.salesforce.com/");
        driver.findElement(By.id("password")).sendKeys("admin123");
        driver.findElement(By.id("Login")).click();

        WebElement errorMessage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("error")))
;

        assertTrue("Error message should be displayed for empty username",
errorMessage.isDisplayed());
    }

    /**

```

```

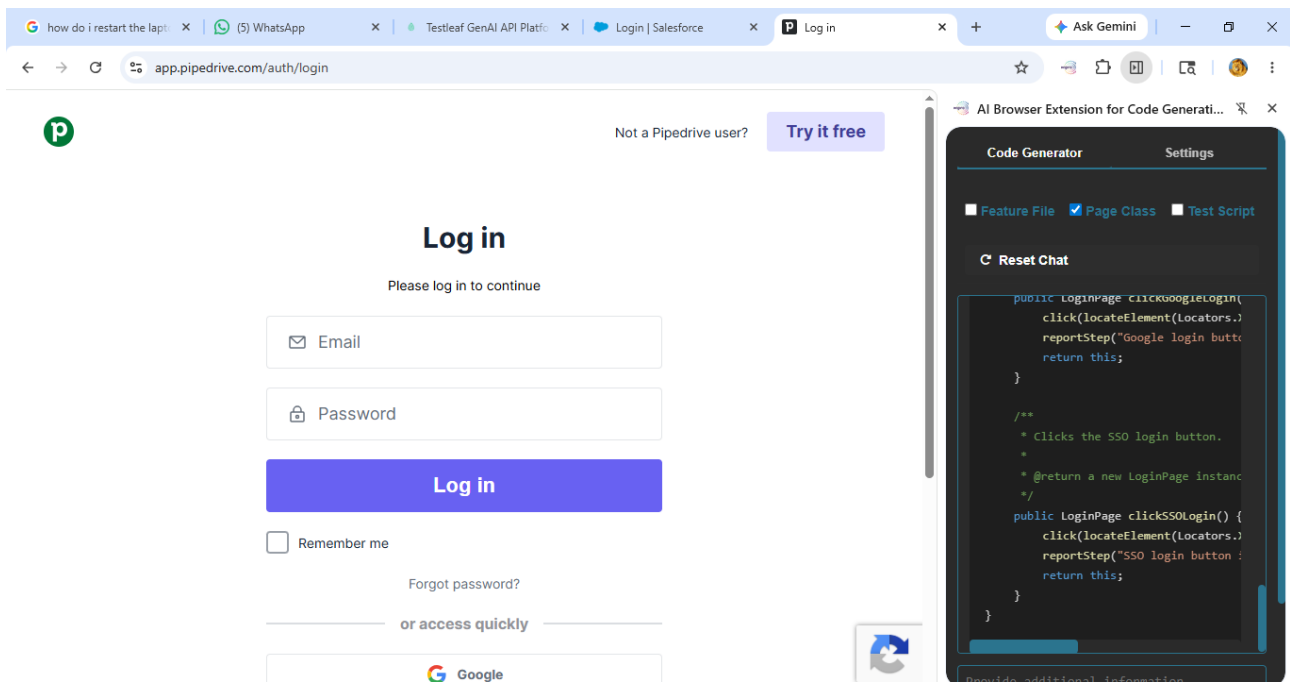
    * Test login with empty password
    */
@Test
public void testLoginWithEmptyPassword() {
    driver.get("https://login.salesforce.com/");
    driver.findElement(By.id("username")).sendKeys("admin");
    driver.findElement(By.id("Login")).click();

    WebElement errorMessage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("error")))
;
    assertTrue("Error message should be displayed for empty password",
errorMessage.isDisplayed());
}

/**
    * Test login with invalid email format
    */
@Test
public void testLoginWithInvalidEmailFormat() {
    driver.get("https://login.salesforce.com/");
    driver.findElement(By.id("username")).sendKeys("invalidemail");
    driver.findElement(By.id("password")).sendKeys("admin123");
    driver.findElement(By.id("Login")).click();

    WebElement errorMessage =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("error")))
;
    assertTrue("Error message should be displayed for invalid email format",
errorMessage.isDisplayed());
}
}

```



Generated Page class

```
package com.testleaf.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import com.testleaf.selenium.api.design.Locators;
import com.testleaf.selenium.api.design.SeleniumBase;

/**
 * Page Object for the Login Page.
 * Provides methods to interact with the login form elements.
 */
public class LoginPage extends SeleniumBase {

    /**
     * Enters the email address in the login form.
     *
     * @param email the email address to enter
     * @return the current LoginPage instance
     */
    public LoginPage enterEmail(String email) {
        clearAndType(locateElement(Locators.XPATH,
"//input[@name='login']")[1]), email);
        reportStep(email + " email is entered successfully", "pass");
        return this;
    }
}
```

```

/**
 * Enters the password in the login form.
 *
 * @param password the password to enter
 * @return the current LoginPage instance
 */
public LoginPage enterPassword(String password) {
    clearAndType(locateElement(Locators.XPATH,
"//input[@name='password'] [1]"), password);
    reportStep(password + " password is entered successfully",
"pass");

    return this;
}

/**
 * Clicks the login button to submit the form.
 *
 * @return a new HomePage instance
 */
public HomePage clickLogin() {
    click(locateElement(Locators.XPATH, "//button[text()='Log
in']"));

    reportStep("Login button is clicked", "pass");
    return new HomePage();
}

/**
 * Clicks the 'Forgot password?' link.
 *
 * @return a new LoginPage instance
 */
public LoginPage clickForgotPassword() {
    click(locateElement(Locators.LINK_TEXT, "Forgot password?"));
    reportStep("Forgot password link is clicked", "pass");
    return this;
}

/**
 * Clicks the 'Remember me' checkbox.
 *
 * @return the current LoginPage instance
 */
public LoginPage clickRememberMe() {

```

```

        click(locateElement(Locators.XPATH,
"//label[@for=':Rjmq9r5:']"));
        reportStep("Remember me checkbox is clicked", "pass");
        return this;
    }

    /**
     * Clicks the Google login button.
     *
     * @return a new LoginPage instance
     */
    public LoginPage clickGoogleLogin() {
        click(locateElement(Locators.XPATH, "//a[@data-track-
action='Login Google']"));
        reportStep("Google login button is clicked", "pass");
        return this;
    }

    /**
     * Clicks the SSO login button.
     *
     * @return a new LoginPage instance
     */
    public LoginPage clickSSOLogin() {
        click(locateElement(Locators.XPATH, "//a[@data-cy='sso-login-
button']"));
        reportStep("SSO login button is clicked", "pass");
        return this;
    }
}

```